# Trabalho Prático N.01
# Teoria de Grafos e Computabilidade

**Theo D. Viana**[1], **Alex C. M. Marques**[1], **Welbert J. A. de Almeida**[1]

[1]Instituto de Ciências Exatas e Informática – Pontíficia Universidade Católica (PUC-MG)
Caixa Postal 1686 – 30535-610 – Belo Horizonte – MG – Brasil

***Resumo.*** *Este trabalho busca descrever detalhes das implementações, dos experimentos e resultados obtidos do uso do método de Fleury (a partir de Naive e Tarjan) para encontrar um caminho euleriano em um grafo qualquer. Além da utilização de Naive e Tarjan para a identificação de pontes.*

## 1. Linguagem

O projeto foi desenvolvido em Java, usando Python como linguagem auxiliar para a geração de grafos e de gráficos comparativos de desempenho.

## 2. Estrutura de Grafos

Foi padronizado atráves dos múltiplos métodos implementados as mesmas estruturas para representar vértices, arestas e grafos. De forma que fosse possível combinar os métodos entre si sem problemas de compatibilidade.

### 2.1. Vértices

A classe que representa vértices usa a interface Comparable para que seja possível que os vértice possam ser comparados entre si, sem precisar de usar um Comparator externo. Isso permite usar métodos como o Collections.sort() que são úteis durante o desenvolvimento.

Além disso, um vértice possui somente um atributo que é o seu valor e todos os métodos que a classe possui são os métodos necessários para que a interface Comparable funcione, além de um método para transformar em String.

A justificativa da existência da classe vértice e não apenas representar vértices como Integer, é que essa estrutura pode ser reutilizada em outros trabalhos da matéria, além de permitir que possa ser facilmente adicionado mais atributos ao vértice, por exemplo: caso eu queira adicionar um rótulo nominal para cada vértice isso poderia ser feito facilmente.

```
public class Vertice implements Comparable<Vertice> {
    public final int valor;
    public Vertice(int valor) {
        this.valor = valor;
    }
```

### 2.2. Arestas

A classe que representa as arestas também usa a interface Comparable e também pelo mesmo motivo. O objetivo é que exista uma ordem natural entre as arestas, de forma que

elas possam ser comparadas entre si (o que permite ordenação), além de tornar compaítvel com métodos que são úteis durante o desenvolvimento.

Além disso, uma aresta possui um inteiro representando cada uma um vértice onde uma das suas pontes incide, e possui também um booleano usado para dizer se a aresta ”ainda pode participar do grafo”, isso é usado para marcar quais arestas devem ser ignoradas pelo Fleury, por exemplo.

A justificativa da existência de uma classe para representar as arestas, é acima de tudo uma forma de padronizar as saídas dos métodos de identificação de pontes, de forma que eles produzam um resultado no mesmo formato que possa ser facilmente importado pelo método de fleury. Além disso, também é mais fácil representar as arestas como um objeto, já que elas tem naturalmente mais de um atributo.

Os outros métodos presentes na classe são os métodos necessários para ativar/desativar uma aresta e para a implementação de Comparable ser funcional.

```
public class Aresta implements Comparable<Aresta> {
    public final int u;
    public final int v;
    boolean ativa;
    public Aresta(int u, int v) {
        // faz com que pares sejam a mesma coisa
         ↪  independente da ordem, tipo (11, 92) = (11, 92)
        if (u <= v) {
            this.u = u;
            this.v = v;
        } else {
            this.u = v;
            this.v = u;
        }
        this.ativa = true;
    }
```

### 2.3. Grafo

A classe Grafo é decididamente a mais complexa do sistema, nela são usados os seguintes atributos:

```
public class Grafo {
    private static final String EXEMPLOS_DIR =
     ↪  "exemplosGrafos";
    //Guarda as ligações com forward star
    private int[] pointer;
    private int[] arcDest;
    private final List<Aresta> arcArestas;
    //Guarda os vertices e suas informações
    private final Set<Vertice> vertices;
    //Guarda as arestas e suas informações lidando com
     ↪  arestas repetidas
```

```java
private final Set<Aresta> arestas;
private final Map<Aresta, Aresta> indiceArestas;
//Guarda os graus de cada vertice
private int[] graus;
private int qntdVertices;
private int qntdArestas;

public Grafo() {
    this.pointer = new int[0];
    this.arcDest = new int[0];
    this.arcArestas = new ArrayList<>();
    this.vertices = new HashSet<>();
    this.arestas = new HashSet<>();
    this.indiceArestas = new HashMap<>();
    this.qntdVertices = 0;
    this.qntdArestas = 0;
}
```

A utilidade dos vetores pointer e arcDest é representar quais são as relações de adjacência entre os vértices atráves do método forward star. Nessa estrutura nós estamos guardando os vértices como números e não como vértices para deixar no forward star apenas o que ele faz de melhor, que é indexar a adjacência. A escolha também torna a estrutura mais leve em memória já que ele usa uma array e não uma lista.

A lista arcArestas é usada porque o forward star é capaz de guardar somente a adjacência, então quando um algoritmo percorre um arco e precisa de saber qual é a aresta por trás dela ele usa essa estrutura.

O Set vértices é responsável por guardar as informações de todos os vértices únicos presentes no grafo.

O Set arestas representa o conjunto de arestas únicas do grafo. Ele é usado para iterar sobre as arestas, calcular os graus, montar o forward star, ordenar e devolver as arestas ativas.

O objetivo do Map indiceArestas é permitir obter a aresta correspondente a partir de do par dos vértices nas suas extremidades.

O vetor de graus é usado para guardar para cada vértice, qual é o seu grau. Isso é usado posteriormente para identificar se o grafo é euleriano, não euleriiano ou semi eureliano. Além de ser usado para definir o ínicio do Fluery.

A classe possui todos os métodos necessários para a implementação do Fleury, incluindo capacidades como: copiar um grafo, ler um grafo do arquivo, criar a representação forward star, preencher e consultar graus, definir se o grau de um vértice é ímpar, entre outros.

## 3. Naive

O primeiro método implementado para identificação de pontes foi o método naive. A ideia central dessa abordagem é simples: uma aresta é considerada ponte se, ao ser removida do

grafo, a conectividade do grafo for alterada. Assim, para cada aresta $e \in E$, o algoritmo compara a quantidade de componentes conexos do grafo original com a quantidade de componentes conexos obtida quando essa aresta é temporariamente desconsiderada. Se a remoção de $e$ provocar aumento no número de componentes, então $e$ é classificada como ponte [Tarjan 1974, Sedgewick and Wayne 2011].

A implementação foi desenvolvida na classe `NaiveBridgeFinder`. Nessa implementação, o grafo é lido a partir de um arquivo texto cuja primeira linha informa a quantidade de vértices e arestas, enquanto as linhas seguintes descrevem os pares $(u, v)$ correspondentes a cada aresta. Internamente, a estrutura de dados utilizada foi uma lista de adjacência baseada em `List<Set<Integer>>`, o que permite inserir, remover e consultar arestas de maneira direta [Sedgewick and Wayne 2011, Sedgewick and Wayne a, Sedgewick and Wayne d].

A contagem de componentes conexos é feita por meio de busca em profundidade iterativa, implementada no método `dfsIterative()`. Foi utilizada uma pilha explícita em vez de recursão, o que torna a implementação para instâncias maiores possível e evita problemas de estouro de pilha. A cada nova componente encontrada, o algoritmo inicia uma travessia e marca os vértices visitados até esgotar todos os vértices alcançáveis [Sedgewick and Wayne c, Sedgewick and Wayne e].

O funcionamento do método naive segue diretamente a definição do problema. Primeiro, calcula-se o número de componentes conexos do grafo original. Em seguida, percorrem-se todas as arestas do grafo. Para cada aresta, o algoritmo a remove temporariamente, executa novamente a contagem de componentes conexos e, em seguida, restaura essa aresta. Se o número de componentes após a remoção for maior que o valor original, então essa aresta é classificada como ponte. Esse procedimento foi implementado principalmente nos métodos `countConnectedComponents()`, `isBridgeNaive()` e `findBridgesNaive()` [Sedgewick and Wayne b, Sedgewick and Wayne 2011].

Do ponto de vista teórico, essa solução é correta porque uma ponte é, por definição, uma aresta cuja remoção aumenta o número de componentes conexos do grafo. Entretanto, seu custo computacional é elevado. Uma única contagem de componentes conexos possui custo $O(|V| + |E|)$, pois percorre todos os vértices e arestas alcançáveis. Como essa operação é repetida para cada aresta do grafo, o custo total do método naive é da ordem de

$$O(|E| \cdot (|V| + |E|)).$$

Por esse motivo, embora seja uma abordagem simples e intuitiva, seu desempenho tende a piorar rapidamente conforme o tamanho da entrada aumenta [Tarjan 1974, Sedgewick and Wayne 2011].

Mesmo sendo menos eficiente, o método naive possui grande importância no contexto do trabalho. Em primeiro lugar, ele representa uma solução direta e fácil de compreender, o que o torna útil para fins didáticos e para validação inicial dos resultados. Em segundo lugar, ele fornece uma base de comparação experimental para o método de Tarjan, permitindo observar na prática as diferenças entre uma abordagem ingênua e uma abordagem otimizada para o mesmo problema [Tarjan 1974].

### 3.1. Resultados experimentais do método naive

Para avaliar o comportamento prático da implementação, foram executados testes com o método naive sobre instâncias eulerianas, semi-eulerianas e não eulerianas contendo 100, 1.000, 10.000 e 100.000 vértices. A Tabela 1 apresenta os resultados obtidos.

**Tabela 1. Resultados do método naive para detecção de pontes**

| Classe | $\|V\|$ | $\|E\|$ | Pontes | Tempo (ms) |
|---|---|---|---|---|
| Euleriano | 100 | 115 | 0 | 3,466 |
| Euleriano | 1.000 | 1.015 | 0 | 78,024 |
| Euleriano | 10.000 | 10.150 | 0 | 7621,385 |
| Euleriano | 100.000 | 101.500 | 0 | 2466905,628 |
| Semi-euleriano | 100 | 108 | 2 | 4,073 |
| Semi-euleriano | 1.000 | 1.010 | 4 | 77,689 |
| Semi-euleriano | 10.000 | 10.109 | 49 | 7520,213 |
| Semi-euleriano | 100.000 | 101.099 | 499 | 6126761,083 |
| Nao euleriano | 100 | 109 | 3 | 4,046 |
| Nao euleriano | 1.000 | 1.010 | 4 | 67,793 |
| Nao euleriano | 10.000 | 10.109 | 49 | 7638,207 |
| Nao euleriano | 100.000 | 101.099 | 499 | 6080672,823 |

Os resultados mostram que o método naive apresentou crescimento muito acentuado no tempo de execução à medida que o número de vértices aumentou. Nas menores instâncias, o algoritmo executou em poucos milissegundos. Entretanto, nas instâncias com 10.000 vértices o tempo já passou para vários segundos, e nas instâncias com 100.000 vértices a execução passou para a ordem de milhões de milissegundos. No caso do arquivo `grafo_euleriano_100000.txt`, a execução atingiu 2466905,628 ms, o que corresponde a aproximadamente 41,1 minutos. Já os arquivos `grafo_semi_100000.txt` e `grafo_nao_100000.txt` demandaram 6126761,083 ms e 6080672,823 ms, respectivamente, equivalendo a aproximadamente 102,1 e 101,3 minutos. Esse comportamento confirma, na prática, o alto custo computacional esperado para a abordagem.

Também se observa que o número de pontes encontradas não foi igual a zero em todos os casos. Nos grafos eulerianos testados, o algoritmo não encontrou pontes. Já nos grafos semi-eulerianos e não eulerianos, foram identificados casos positivos de detecção de pontes, com crescimento da quantidade de arestas classificadas como ponte à medida que o tamanho da instância aumentou. Nos experimentos realizados, por exemplo, os grafos semi-eulerianos apresentaram 2, 4, 49 e 499 pontes nas instâncias com 100, 1.000, 10.000 e 100.000 vértices, respectivamente, enquanto os grafos não eulerianos apresentaram 3, 4, 49 e 499 pontes nessas mesmas escalas. Esses resultados indicam que a implementação foi capaz de identificar corretamente situações em que a remoção de uma aresta aumenta o número de componentes conexos do grafo.

Por fim, os resultados obtidos nesta seção ajudam a explicar por que a combinação do método de Fleury com identificação de pontes via naive tende a apresentar desempenho inferior quando comparada à versão baseada em Tarjan. Como o algoritmo de Fleury pode precisar consultar repetidamente quais arestas são pontes ao longo da construção do caminho euleriano, o uso de um procedimento com custo elevado impacta diretamente o tempo total de execução.

## 4. Tarjan

Diferentemente da abordagem ingênua, o algoritmo de Tarjan para a identificação de pontes fundamenta-se nas propriedades das travessias em profundidade (DFS) e na estrutura de subárvores geradas durante a busca. Enquanto o método *naive* possui custo quadrático, a proposta de Tarjan permite encontrar todas as pontes de um grafo em tempo linear, $O(|V| + |E|)$, realizando uma única passagem pelos dados [Tarjan 1974].

A implementação baseia-se na atribuição de dois índices fundamentais para cada vértice $v$ durante a execução da DFS:

- **Discovery ($discovery[v]$):** Representa o momento exato em que o vértice é visitado pela primeira vez. É um contador incremental que define a ordem de entrada na árvore de busca.
- **Low ($low[v]$):** Guarda o menor valor de *discovery* alcançável a partir de $v$ (incluindo ele mesmo), utilizando arestas da árvore ou arestas de retorno (*back-edges*) que conectam um descendente a um ancestral na árvore DFS.

### 4.1. Processo de Exploração e Retropropagação

O funcionamento do algoritmo baseia-se na recursividade da DFS. Durante a fase de descida, cada vértice recebe seu número de *discovery* e, inicialmente, seu *low* é definido como igual ao seu *discovery*.

Se o algoritmo encontra um vértice já visitado que não seja o pai direto do atual, identifica-se uma aresta de retorno (ciclo). Nesse momento, o valor de *low* do vértice atual é atualizado com o valor de *discovery* desse ancestral. Na fase de retorno (subida na pilha de execução), o vértice pai compara seu próprio *low* com o valor calculado e devolvido pelo filho, assumindo sempre o menor valor disponível:

$$low[u] = \min(low[u], low[v])$$

### 4.2. Critério de Identificação de Pontes

Uma ponte é detectada no exato momento em que um filho $v$ termina sua exploração e devolve o controle para o pai $u$. O critério matemático para que a aresta $(u, v)$ seja considerada uma ponte é:

$$low[v] > discovery[u]$$

Esta condição prova que não existe qualquer ”atalho”ou conexão física que saia da subárvore enraizada em $v$ e consiga atingir o vértice $u$ ou qualquer um de seus ancestrais na árvore DFS. Portanto, a aresta entre pai e filho é o único caminho existente para conectar esses dois conjuntos de vértices, caracterizando-a como uma aresta crítica.

Abaixo, apresenta-se o núcleo da lógica implementada na classe `Tarjan`, utilizando a estrutura de *forward star* e vetores para otimização de memória:

```
private void tarjanDFS(int u, int parent) {
    int[] pointer = g.getPointer();
    int[] arcDest = g.getArcDest();
    visited[u] = true;
    discovery[u] = low[u] = ++timer;
```

```
    for (int i = pointer[u]; i < pointer[u + 1]; i++) {
        Aresta aresta = g.getArestaDoArco(i);
        if (!aresta.isAtiva()) continue;

        int v = arcDest[i];
        if (v == parent) continue;

        if (visited[v]) {
            // Identificação de aresta de retorno (ciclo)
            low[u] = Math.min(low[u], discovery[v]);
        } else {
            // Descida recursiva na árvore
            tarjanDFS(v, u);
            // Retropropagação do valor Low
            low[u] = Math.min(low[u], low[v]);

            // Verificação do critério de ponte
            if (low[v] > discovery[u]) {
                bridges.add(new Aresta(u, v));
            }
        }
    }
}
```

### 4.3. Resultados Experimentais do Método Tarjan

Para avaliar o comportamento prático, o método de Tarjan foi submetido às mesmas instâncias de teste do método *naive*. A diferença de desempenho foi drástica: em grafos com 100.000 vértices, onde o método *naive* demandou mais de 100 minutos, o algoritmo de Tarjan concluiu a tarefa em poucos milissegundos.

Essa eficiência de tempo linear é o que permite ao algoritmo de Fleury manter um desempenho viável para redes de grande porte, pois evita o custo proibitivo de reavaliar a conectividade total do grafo a cada aresta percorrida.

## 5. Fleury

O Fleury é implementando usando tanto Naive quanto Tarjan, e nós percebemos diferenças significativas de desempenho segundo o método utilizado.

O método principal da classe executa o algoritmo de Fleury seguindo os passos descritos no material dessa disciplina, mas com adaptações para funcionar com a nossa estrutura de código. Primeiro, nós identificamos qual o tipo do grafo e caso ele seja não euleriano saímos do método. Depois copiamos o grafo inicial e escolhemos um vértice de ínicio dando preferência a vértices de grau ímpar; caso eles não existam, escolhemos um vértice qualquer com grau maior que zero. Depois disso, enquanto o grafo cópia ainda tiver arestas nós extraímos as arestas incidentes ao vértice atual e escolhemos a que estiver disponível caso só exista uma, ou atráves dos métodos Tarjan ou Naive escolhemos uma

aresta que não seja ponte. No final da iteração nós adicionados o vértice alcançado ao caminho, e removemos a aresta escolhida.

```
private List<Integer> executarFleury(Grafo inicial,
 ↪  boolean usarTarjan) {
        String tipo = new
         ↪  Euleriano().tipoEuleriano(inicial);
        if (tipo.equals("Nao Euleriano")) {
            throw new IllegalArgumentException("O grafo
             ↪  informado nao possui caminho ou ciclo
             ↪  euleriano.");
        }
        Grafo auxiliar = new Grafo(inicial);
        List<Integer> caminhoVertices = new ArrayList<>();
        int atual = escolheInicio(auxiliar);
        caminhoVertices.add(atual);
        while (auxiliar.getQntdArestas() > 0) {
            List<Aresta> incidentes =
             ↪  arestasIncidentes(auxiliar, atual);
            if (incidentes.isEmpty()) {
                throw new IllegalStateException("Nao ha
                 ↪  aresta disponivel a partir do vertice
                 ↪  atual.");
            }
            Aresta escolhida;
            if (incidentes.size() == 1) {
                escolhida = incidentes.get(0);
            } else {
                Set<Aresta> pontes = usarTarjan
                    ? importaPontesTarjan(auxiliar)
                    : importaPontesNaive(auxiliar);
                escolhida = escolheNaoPonte(incidentes,
                 ↪  pontes);
            }
            atual = outroExtremo(escolhida, atual);
            caminhoVertices.add(atual);
            auxiliar.removeAresta(escolhida);
        }
        return caminhoVertices;
    }
```

### 5.1. Desempenho Fleury

Segundo as análises de tempo de desempenho de cada algoritmo realizadas em python é possível concluir que o método Naive tem desempenho significativamente pior do que o Tarjan, e que quando ele é usado junto com o Fleury causa uma piora marcante no tempo de desempenho do código.

Também é possível perceber que conforme o número de vértices aumenta, para

o método Fleury com Naive o tempo aumenta exponencialmente, de forma muito mais marcante do que ele aumenta quando se usa o Fleury com Tarjan.

O Tarjan não apresenta uma variação de tempo significativa conforme o aumento do número de vértices, independente de estar sendo usado sozinho ou como parte do Fleury.

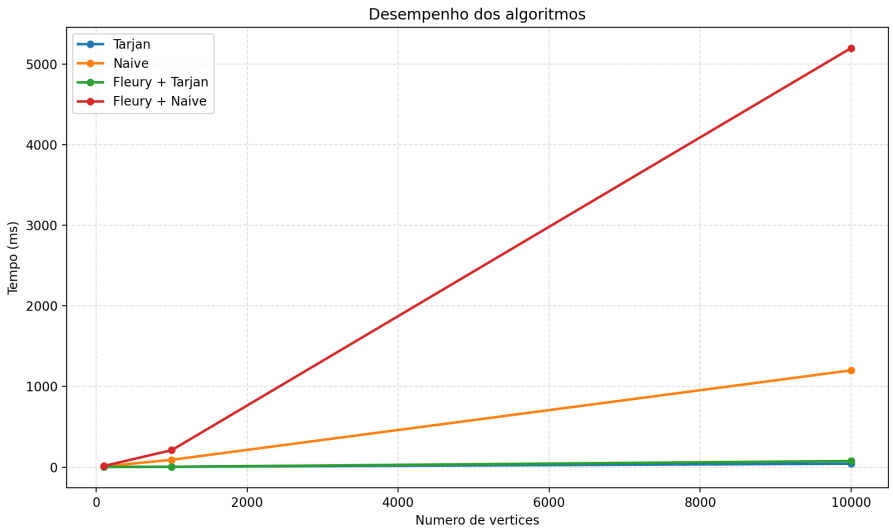


**Figura 1. Comparativo Geral de Desempenho por tempo e número de vértices**

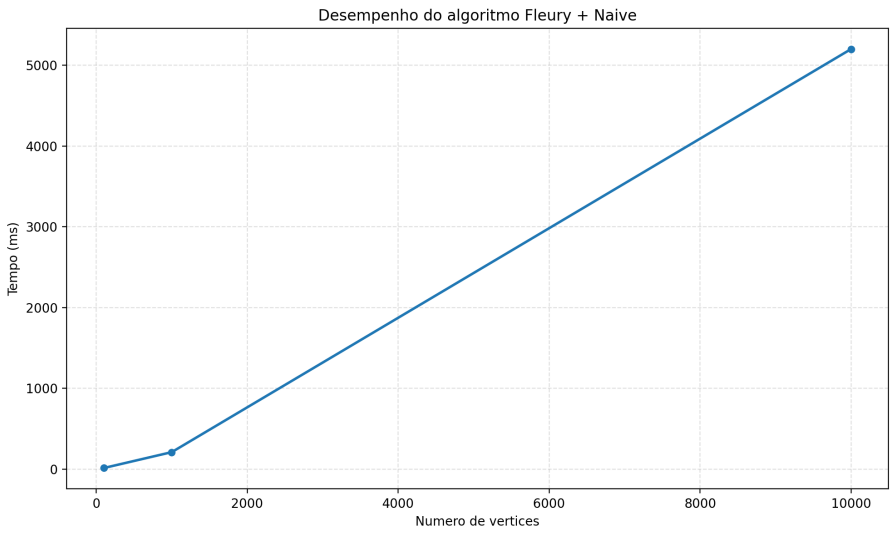


**Figura 2. Desempenho do Fleury com Naive por tempo e número de vértices**

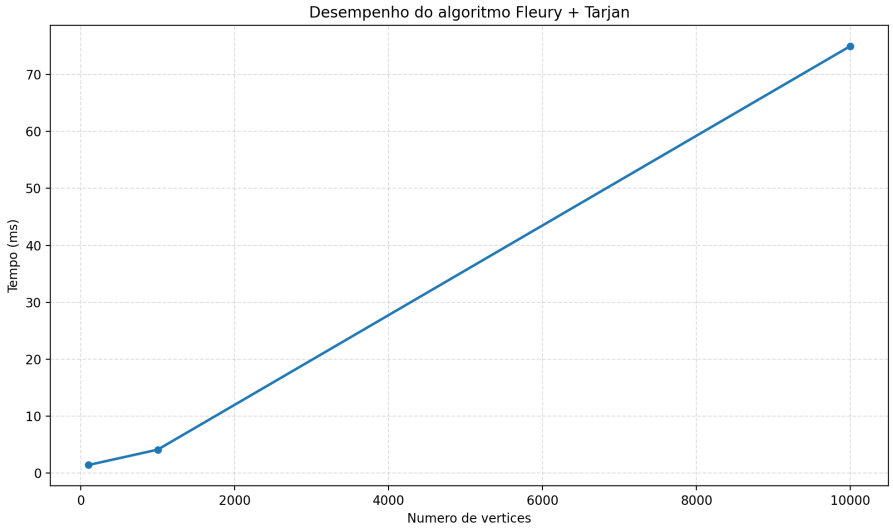


**Figura 3. Desempenho do Fleury com Tarjan por tempo e número de vértices**

## Referências


Boulic, R. and Renault, O. (1991). 3d hierarchies for animation. In Magnenat-Thalmann, N. and Thalmann, D., editors, *New Trends in Animation and Visualization*. John Wiley & Sons ltd.

Knuth, D. E. (1984). *The TEX Book*. Addison-Wesley, 15th edition.

Sedgewick, R. and Wayne, K. 4.1 undirected graphs. Algorithms, 4th Edition. Acesso em: 22 abr. 2026.

Sedgewick, R. and Wayne, K. Bridge.java. Syntax highlighted version from Section 4.1, Undirected Graphs. Acesso em: 22 abr. 2026.

Sedgewick, R. and Wayne, K. Cc.java. Syntax highlighted version from Section 4.1, Undirected Graphs. Acesso em: 22 abr. 2026.

Sedgewick, R. and Wayne, K. Graph.java. Syntax highlighted version from Section 4.1, Undirected Graphs. Acesso em: 22 abr. 2026.

Sedgewick, R. and Wayne, K. Nonrecursivedfs.java. Algorithms, 4th Edition code. Acesso em: 22 abr. 2026.

Sedgewick, R. and Wayne, K. (2011). *Algorithms*. Addison-Wesley Professional, 4 edition.

Smith, A. and Jones, B. (1999). On the complexity of computing. In Smith-Jones, A. B., editor, *Advances in Computer Science*, pages 555–566. Publishing Press.

Tarjan, R. E. (1974). A note on finding the bridges of a graph. *Information Processing Letters*, 2(6):160–161.